

Sieve of Eratosthenes

Last Updated : 23 Jul, 2025



Given a number n , find all prime numbers less than or equal to n .

Examples:

Input: $n = 10$

Output: $[2, 3, 5, 7]$

Explanation: The prime numbers up to 10 obtained by Sieve of Eratosthenes are $[2, 3, 5, 7]$.

Input: $n = 35$

Output: $[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]$

Explanation: The prime numbers up to 35 obtained by Sieve of Eratosthenes are $[2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31]$.

Table of Content

- [\[Naive Approach\] - Using Loop \$O\(n \cdot \sqrt{n}\)\$ Time and \$O\(1\)\$ Space](#)
- [\[Efficient Approach\] - Sieve of Eratosthenes](#)

[Naive Approach] - Using Loop $O(n \cdot \sqrt{n})$ Time and $O(1)$ Space

The Naive Approach for finding all prime numbers from 1 to n involves checking each number individually to determine whether it is prime.

Step By Step Implementations:

- Loop through all numbers i from 2 to n .
- For each i , check if it is divisible by any number from 2 to $i - 1$.
- If it is divisible, then i is not prime.
- If it is not divisible by any number in that range, then i is prime.

[Efficient Approach] - Sieve of Eratosthenes

The Sieve of Eratosthenes efficiently finds all primes up to n by repeatedly marking multiples of each prime as non-prime, starting from 2. This avoids

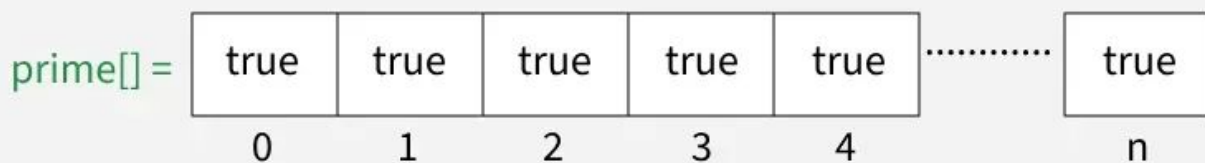
redundant checks and quickly filters out all composite numbers.

Step By Step Implementations:

- Initialize a Boolean array `prime[0..n]` and set all entries to `true`, except for `0` and `1` (which are not primes).
- Start from `2`, the smallest prime number.
- For each number `p` from `2` up to $\sqrt{n}$:
 - If `p` is marked as `prime(true)`:
 - Mark all multiples of `p` as not prime(`false`), starting from `p * p` (since smaller multiples have already been marked by smaller primes).
- After the loop ends, all the remaining `true` entries in `prime` represent prime numbers.

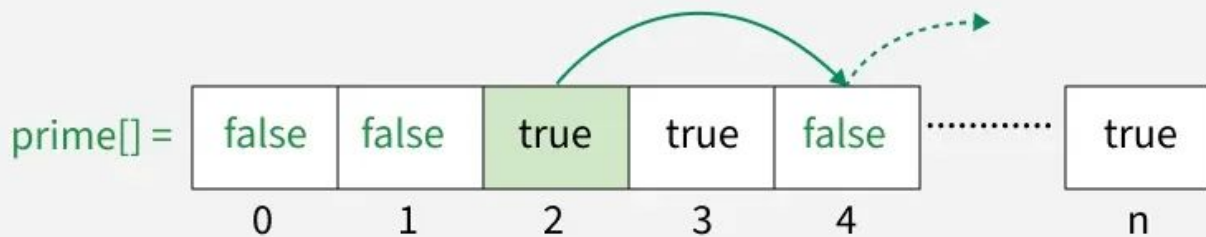
01
Step

Initialize an array `prime` of size `n + 1`, with all entries set to `true`, assuming all numbers are prime initially.



02
Step

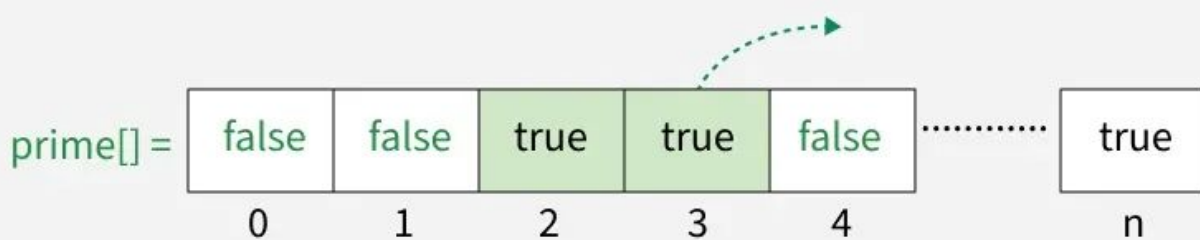
Start with $p = 2$, the first prime number, and for each p , eliminate all its multiples from the list of primes.



Note : 0 and 1 are non prime numbers, so they are marked with false.

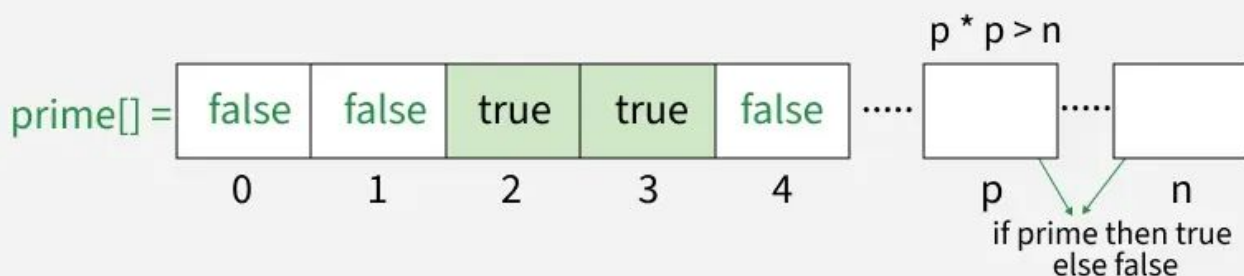
03
Step

After completing elimination with $p = 2$, move to $p = 3$. Since it's still marked as prime, eliminate all its multiples.



04
Step

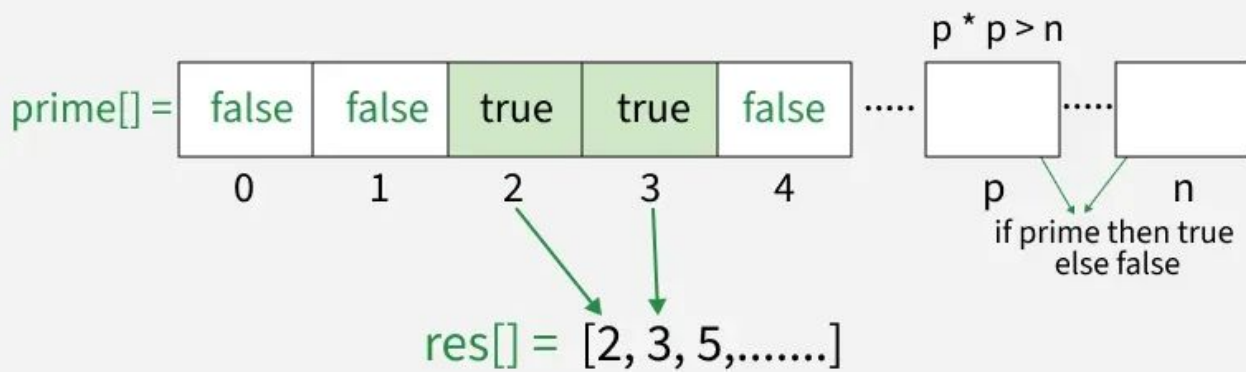
In the same way, we repeat this step for all numbers still marked as true, continuing the process while $p^2 \leq n$



At this point, all non-prime numbers have been marked as false.

05
Step

Traverse the prime[] array from 2 to n. For each index marked true, add it to the result array res(which will contain all the prime numbers).



C++

Java

Python

C#

JavaScript

```
#include <iostream>
#include <vector>
using namespace std;

vector<int> sieve(int n) {

    // creation of boolean array
    vector<bool> prime(n + 1, true);
```

```

for (int p = 2; p * p <= n; p++) {
    if (prime[p] == true) {

        // marking as false
        for (int i = p * p; i <= n; i += p)
            prime[i] = false;
    }
}

vector<int> res;
for (int p = 2; p <= n; p++){
    if (prime[p]){
        res.push_back(p);
    }
}
return res;
}

int main(){
    int n = 35;
    vector<int> res = sieve(n);
    for(auto ele : res){
        cout << ele << ' ';
    }
    return 0;
}

```

Output

```
2 3 5 7 11 13 17 19 23 29 31
```

Time Complexity: $O(n \cdot \log(\log(n)))$. For each prime number, we mark its multiples, which takes around n/p steps. The total time is proportional to $n \cdot (1/2 + 1/3 + 1/5 + \dots)$.

This sum over primes grows slowly and is approximately $O(n \cdot \log(\log(n)))$ making the algorithm very efficient.

Auxiliary Space: $O(n)$

Related articles

- [How is the time complexity of Sieve of Eratosthenes is \$n \cdot \log\(\log\(n\)\)\$?](#)

- [Segmented Sieve.](#)
- [Sieve of Eratosthenes in \$O\(n\)\$ time complexity](#)
- [Sieve of Sundaram](#)
- [Sieve of Atkin](#)

Sieve of Eratosthenes

[Visit Course ↗](#)[Comment](#)[More info ▼](#)[Advertise with us](#)

Similar Reads

[Basics & Prerequisites](#)[Data Structures](#)[Algorithms](#)[Advanced](#)[Interview Preparation](#)[Practice Problem](#)

Do Not Sell or Share My Personal Information